Kcalm — Struttura del progetto

Organizzazione delle cartelle secondo il pattern **MVC** della Lezione 2 (Architettura App) del corso di Ambienti di Programmazione e Programmazione Mobile.

1 · L'idea in una pagina

Il codice è diviso in **due grandi cartelle** dentro `lib/`:

- `UI/` → la **View**: tutto quello che l'utente vede e tocca (pagine, widget, tema, lingue).
- `model/` → **Model + Controller**: i dati, la logica e l'accesso al "mondo esterno" (database, internet, preferenze, sensori).

Ogni livello parla **solo con quello subito sotto**. Le frecce vanno sempre verso il basso:

UI/pages · UI/widgets	Le schermate e i pezzi riutilizzabili. Mostrano lo stato e, quando l'utente fa qualcosa, chiamano un metodo del provider.

▼ ref.watch(...) per leggere · ref.read(...notifier).metodo() per agire

model/providers	Lo **stato** dell'app con Riverpod (Notifier). Un unico stato per argomento, modificabile solo tramite i suoi metodi.

▼

model/Model.dart	Il "**controller**": singleton, unico punto d'accesso ai dati. Coordina i manager (es. salva sul DB e restituisce il risultato).

▼

model/managers	Parlano con l'esterno: **DatabaseManager** (SQLite), **RestManager** (Open Food Facts), **LocalStorageManager** (preferenze).

▼

Esterno	SQLite · API REST · SharedPreferences · (più avanti: sensori, notifiche, GPS)

model/objects	Le classi dei dati (freezed). Le usano **tutti** i livelli.	**model/support**	Costanti, calcoli puri, extension. Le usano **tutti** i livelli.

Perché così?

Se un giorno passi da dati in memoria a SQLite (o da SQLite a Firestore) cambi **solo i manager e il Model**: le pagine non se ne accorgono. È esattamente il tipo di scelta da raccontare all'orale.

2 · L'albero completo delle cartelle

In **verde** le cartelle, in grigio chiaro i file da aggiungere più avanti (quando arrivano le lezioni su sensori e notifiche). Nomi dei file in PascalCase, come negli esempi del prof.

```
kcalm/
├── lib/
│   ├── main.dart                       avvio: runApp(ProviderScope(child: App()))
│   ├── UI/                             ── VIEW: tutto ciò che si vede
│   │   ├── App.dart                    MaterialApp: temi, lingue, home: Layout()
│   │   ├── pages/                      schermate intere (StatefulWidget)
│   │   │   ├── Layout.dart             NavigationBar in basso + le 4 pagine
│   │   │   ├── DiaryPage.dart
│   │   │   ├── AddFoodPage.dart
│   │   │   ├── WeightPage.dart
│   │   │   ├── ProfilePage.dart
│   │   │   └── onboarding/
│   │   │       ├── WelcomePage.dart
│   │   │       ├── GoalPage.dart
│   │   │       └── AboutYouPage.dart
│   │   ├── widgets/                    custom widget riutilizzabili (Lezione 3)
│   │   │   ├── CalorieRing.dart        anello animato con TickerProviderStateMixin
│   │   │   ├── MealSection.dart        card di un pasto con la lista degli alimenti
│   │   │   ├── MacroBar.dart           barra carboidrati / proteine / grassi
│   │   │   ├── FoodDetailSheet.dart    bottom sheet con grammi e kcal
│   │   │   └── WeightChart.dart        grafico del peso
│   │   ├── behaviors/                  comportamenti trasversali della UI
│   │   │   └── AppLocalizations.dart   legge localizable/*.json (Lezione 3)
│   │   └── aspects/                    aspetto grafico
│   │       └── AppTheme.dart           ThemeData chiaro e scuro, Material 3
│   └── model/                          ── MODEL + CONTROLLER: dati e logica
│       ├── Model.dart                  singleton: unico punto d'accesso ai dati
│       ├── providers/                  stato Riverpod (Notifier + classe di stato)
│       │   ├── ProfileProvider.dart
│       │   ├── DiaryProvider.dart
│       │   ├── WeightProvider.dart
│       │   └── SettingsProvider.dart   tema e lingua scelti
│       ├── managers/                   accesso al "mondo esterno"
│       │   ├── DatabaseManager.dart    SQLite: tabelle, query
│       │   ├── LocalStorageManager.dart SharedPreferences: tema, lingua, onboarding fatto
│       │   ├── RestManager.dart        HTTP verso Open Food Facts
│       │   ├── StepCounterManager.dart  (più avanti — lezione sensori)
│       │   └── NotificationManager.dart (più avanti — lezione notifiche)
│       ├── objects/                    classi dei dati (freezed)
│       │   ├── UserProfile.dart
│       │   ├── Food.dart
│       │   ├── DiaryEntry.dart
│       │   ├── WeightEntry.dart
│       │   └── Enums.dart              MealType, Goal, ActivityLevel, Sex
│       └── support/                    utilità senza stato, Dart puro
│           ├── Constants.dart          nome app, URL API, valori fissi
│           ├── BudgetCalculator.dart   formula Mifflin-St Jeor
│           ├── MealTimeResolver.dart   ora → colazione / pranzo / cena
│           └── extensions/
│               └── DateTimeExtensions.dart
├── localizable/                        traduzioni (dichiarate come assets nel pubspec)
│   ├── it.json
│   └── en.json
├── test/                               stessa struttura di lib/
│   └── model/support/
│       └── BudgetCalculator_test.dart  i test DEVONO finire con _test.dart
├── pubspec.yaml
└── analysis_options.yaml
```

3 · "Questo file dove lo metto?"

Se devi scrivere…	Cartella	Esempio in Kcalm
Una schermata intera, che si raggiunge dalla barra o con `Navigator.push`	`UI/pages/`	`WeightPage`, `GoalPage`
Un pezzo di interfaccia usato in più punti, o abbastanza grande da meritare un file	`UI/widgets/`	`CalorieRing`, `MealSection`
Colori, font, forme dei componenti	`UI/aspects/`	`AppTheme`
Il codice che carica le traduzioni	`UI/behaviors/`	`AppLocalizations`
Un testo visibile all'utente	`localizable/*.json`	`"diary": "Diario"`
Uno stato che la UI deve osservare e che cambia nel tempo	`model/providers/`	pasti del giorno, peso attuale
Un'operazione che coordina più cose (es. "aggiungi pasto" = salva su DB + aggiorna il totale)	`model/Model.dart`	`addDiaryEntry(...)`
Lettura/scrittura su database, internet, preferenze, sensori	`model/managers/`	`RestManager.searchFood(...)`
Una classe che rappresenta un dato	`model/objects/`	`Food`, `WeightEntry`
Un calcolo che dipende solo dai parametri (niente UI, niente DB)	`model/support/`	`BudgetCalculator`
Un metodo in più su una classe esistente	`model/support/extensions/`	`DateTime.isSameDay()`

Le regole da non rompere

- Una **pagina o un widget non importa mai** `sqflite`, `http` o `shared_preferences`: chiede tutto ai provider.
- Un **manager non importa mai** `package:flutter/material.dart`: non sa nulla di widget.
- `objects/` e `support/` sono **Dart puro** → si testano senza emulatore.
- Niente colori o testi scritti a mano nei widget: colori da `Theme.of(context)`, testi da `AppLocalizations`.
- Evita `!` sui nullable: usa `?.`, `??`, `??=` (Lezione 1, Null Safety).

Model.dart e providers: che differenza c'è?

Model.dart — il "controller"

Fa il lavoro: decide quale manager chiamare e in che ordine. Non tiene lo stato della UI. È un singleton (`Model.instance`).

providers — lo stato

Tengono i dati che la UI mostra. Quando cambiano, le pagine che li osservano si ricostruiscono da sole. Chiamano il Model per fare il lavoro.

4 · Esempio: cosa succede quando aggiungi un alimento

L'utente è nel foglio dettaglio di "Penne rigate", sceglie 80 g e preme **Aggiungi al pranzo**.

1. **UI** `FoodDetailSheet` chiama `ref.read(diaryProvider.notifier).addEntry(food, 80, MealType.lunch)` e chiude il foglio.
2. **provider** `DiaryProvider.addEntry` chiama `Model.instance.addDiaryEntry(...)`, poi aggiorna il proprio `state` con la nuova lista dei pasti.
3. **Model** `Model.addDiaryEntry` salva l'alimento in cache (`foods`) se non c'è già, poi chiede al manager di inserire la voce.
4. **manager** `DatabaseManager.insertDiaryEntry` esegue l' `INSERT` nella tabella `diary_entries` di SQLite.
5. **UI** `DiaryPage` fa `ref.watch(diaryProvider)` : vede il nuovo stato, si ricostruisce e `CalorieRing` anima l'anello verso il nuovo valore.

> **Nota**
>
> Finché non arriva la lezione su SQLite, al punto 4 il Model può tenere i dati in una lista in memoria. Quando passi a SQLite cambi solo il punto 3–4: i punti 1, 2 e 5 restano identici.

5 · Come creare le cartelle

Da terminale (nella cartella `kcalm`)

```
# tutte le cartelle in un colpo solo
mkdir -p lib/UI/pages/onboarding \
  lib/UI/widgets lib/UI/behaviors \
  lib/UI/aspects lib/model/providers \
  lib/model/managers lib/model/objects \
  lib/model/support/extensions \
  localizable test/model/support
```

In alternativa: in IntelliJ, tasto destro su `lib` → **New** → **Directory**.

> **Attenzione**
>
> Git **non salva le cartelle vuote**. Prima del commit crea almeno un file in ognuna (anche una classe vuota), altrimenti su GitHub non compaiono.

pubspec.yaml

```
flutter:
  uses-material-design: true
  assets:
    - localizable/
```

analysis_options.yaml

```
include: package:flutter_lints/flutter.yaml
linter:
  rules:
    file_names: false   # PascalCase come il prof
    prefer_const_constructors: true
    avoid_print: true
```

Checklist del primo commit

- ☐ Cartelle create, ognuna con almeno un file
- ☐ `main.dart` avvia `App()`; `App` ha `home: Layout()`
- ☐ `Layout` con `NavigationBar` e 4 pagine vuote (un `Text` al centro)
- ☐ `it.json` / `en.json` con i nomi delle 4 tab, letti da `AppLocalizations`
- ☐ `flutter analyze` senza errori → **pull** → commit "*Struttura MVC, tema e navigazione*" → push

Riferimenti: Lezione 1 (Git, Null Safety, Freezed, Isolates) · Lezione 2 (Architettura App, Progettazione UI) · Lezione 3 (Custom Widget, Localization, Layout, Navigation, Themes, Animations) · Lezione 4 (Gestione dello stato: setState, Riverpod).